Robot Dynamics & Control

2026

Session 4 – Dynamic Robot Control

Tutors:
Pedro Luis Figueroa Saire
Danial Sabzevari

Emails: name.surname@edu.unige.it (or Teams)

Dibris

UNIVERSITÀ DEGLI STUDI
DI GENOVA

Outline

- Use the Matlab **Robotics System Toolbox**
- Use **toolbox blocks** with **Simscape Multibody designs**
- Import a **commercial robot** model **from the toolbox to Simscape**
- Implement **PD control** for **regulation**
- Implement **Computed Torque** for **tracking**

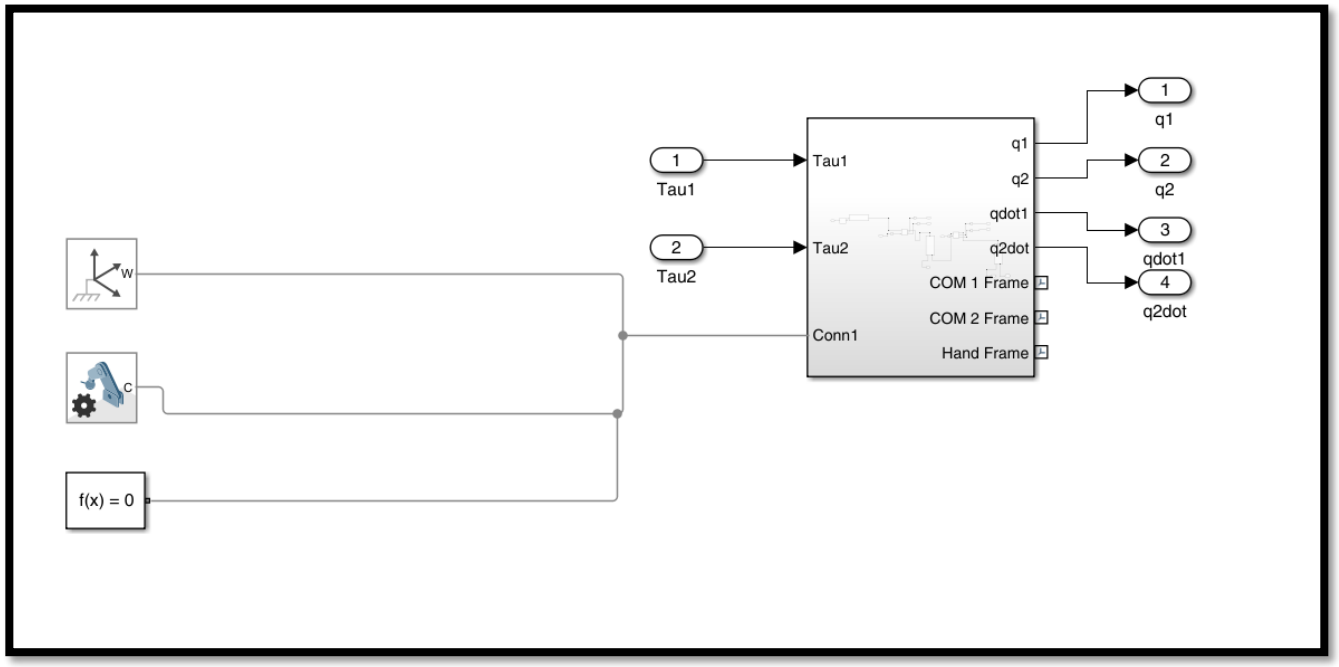
Using your own models

The **Robotics System Toolbox** provides **Simulink blocks** for **dynamic computation (Mass Matrix, Coriolis terms, Gravity Torques)**

A **'Rigid Body Tree'** entity is needed

For a model defined from scratch in **Simscape Multibody** it's straightforward

1° Step: Create you mechanism (Example 2R Planar robot)



Configure the gravity [0,0,-9.81] in the 'Mechanism Configuration block'

Save the file as: 'e4_pendulum.slx'

Hint: Make a copy of your model which you don't edit and use it just for loading the tree (Example: planar_2r.slx)

UNIVERSITÀ DEGLI STUDI DI GENOVA

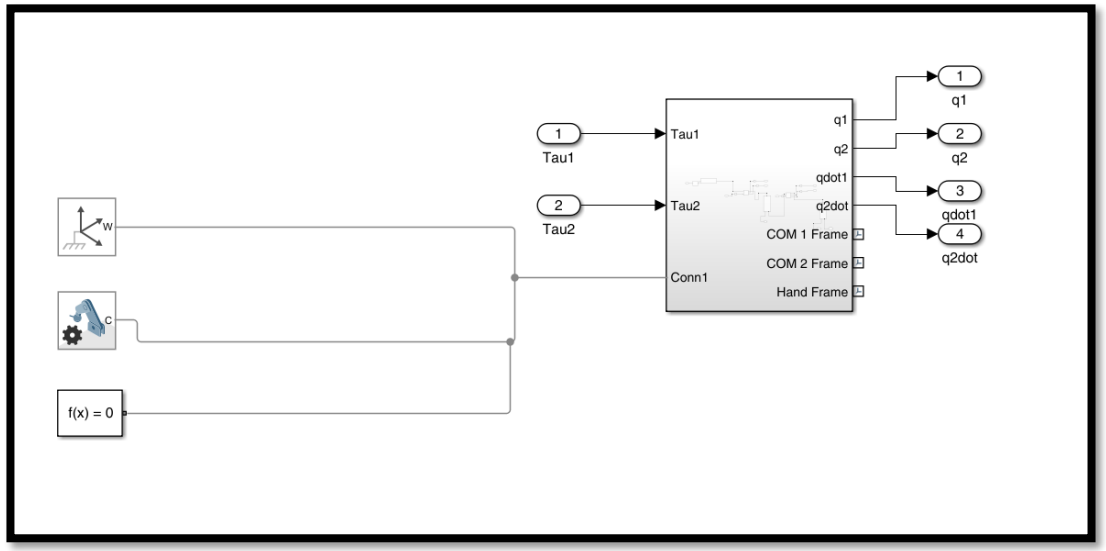
Using your own models

2° Step: In the workspace, load your mechanism as a rigid body tree

```
[robot_2r,importInfo_2r] = importrobot('e4_double_pendulum.slx');
```

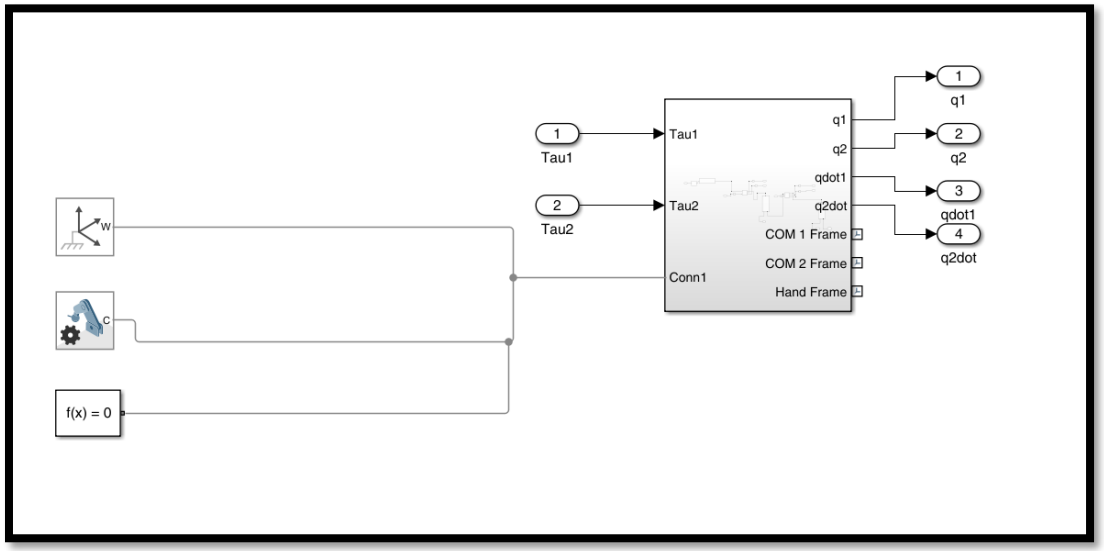
Your workspace should look like this

importInfo_2r	*1×1 rigidBodyTreeImportInfo*	1×1	rigidBodyTreeImportInfo
robot_2r	*1×1 rigidBodyTree*	1×1	rigidBody Tree



'e4_double_pendulum.slx'

.Slx file containing your rigid body tree. If a modification is required (Ex. Add an extra mass). Modify the file, save it, and rerun 'importrobot'



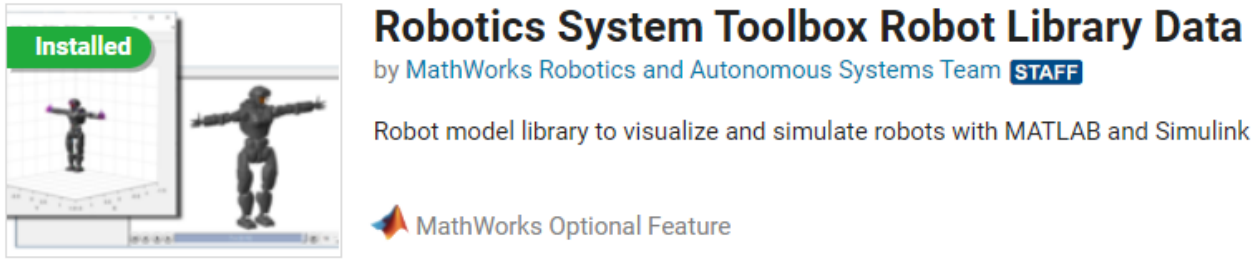
'planar_2r.slx'

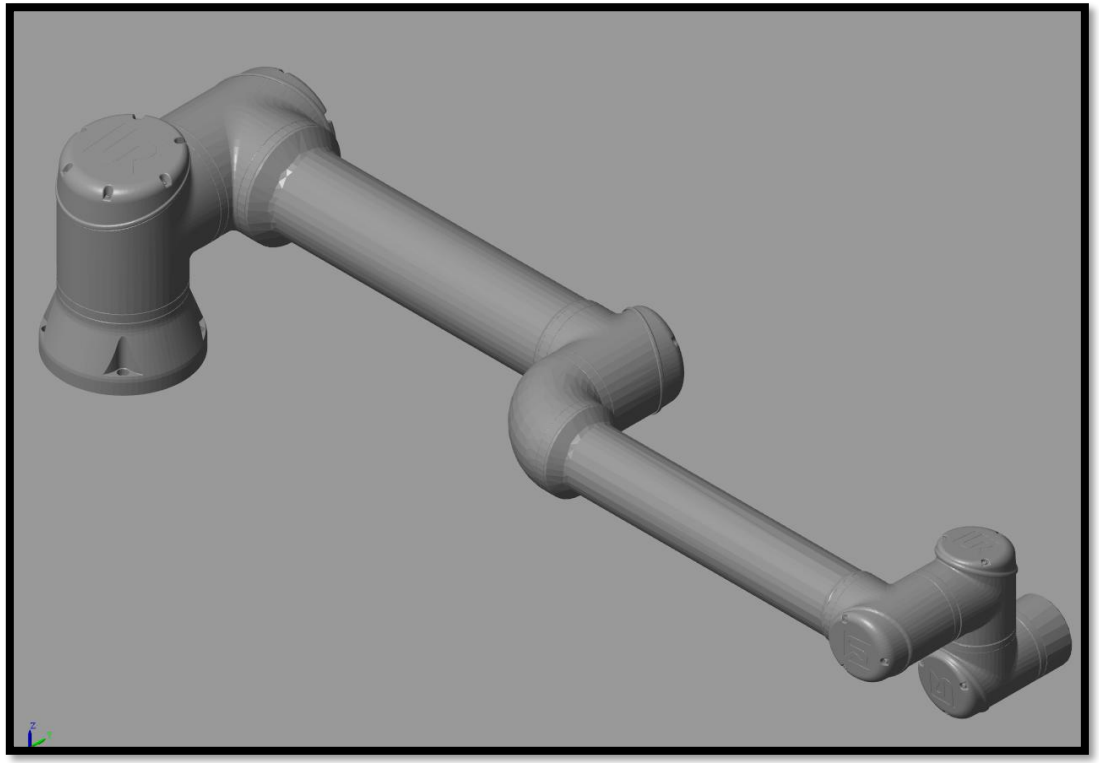
.Slx file containing the mechanism to simulate/control and call the dynamic computation blocks

Using commercial robots

You can use built-in robots (which have **their own rigid body trees**) to define a complete Simscape Multibody model

To correctly visualize the robot you need to install the **Robot Library Data**





Using commercial robots

You can use built-in robots (which have **their own rigid body trees**) to define a complete Simscape Multibody model

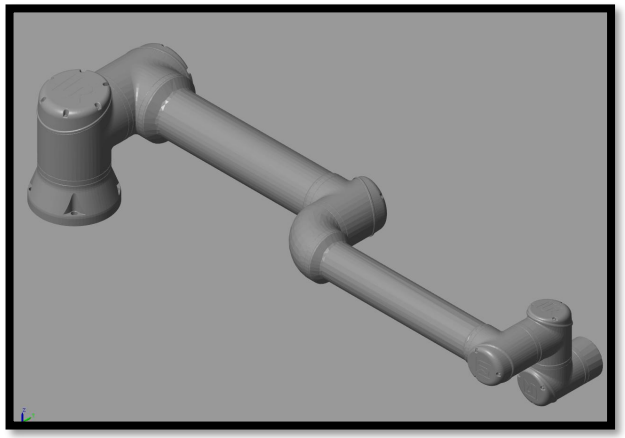
1° Step: Load from the toolbox(Example UR Robot)-> Save .slx model (Rigid body Tree), make copy (Simulation)

```
% Load from toolbox
robot=loadrobot("universalUR10e",DataFormat="column");
% Set Gravity in the mechanism
robot.Gravity=[0,0,-9.81];
% Build a new multibody model
robotSM=smimport(robot,ModelName="UR10_model");
```

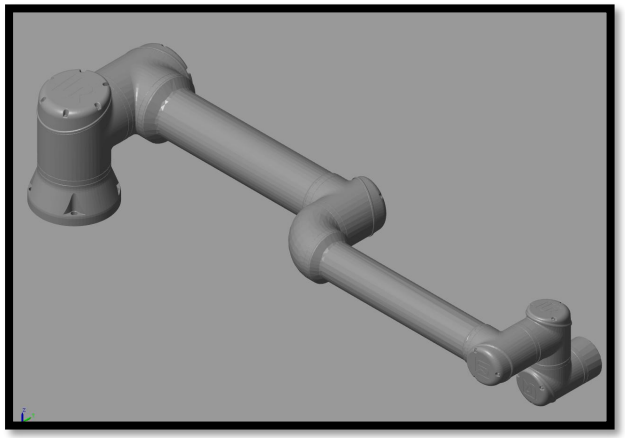
2° Step: use import_robot to load 'UR10_model.slx'

```
>> [robot_UR10,importInfo_UR10]=importrobot('UR10_model.slx')
```

importInfo_UR10	*1×1 rigidBodyTreeImportInfo*	1×1	rigidBodyTreeImportInfo
robot_UR10	*1×1 rigidBodyTree*	1×1	rigidBodyTree



UR10_model.slx'

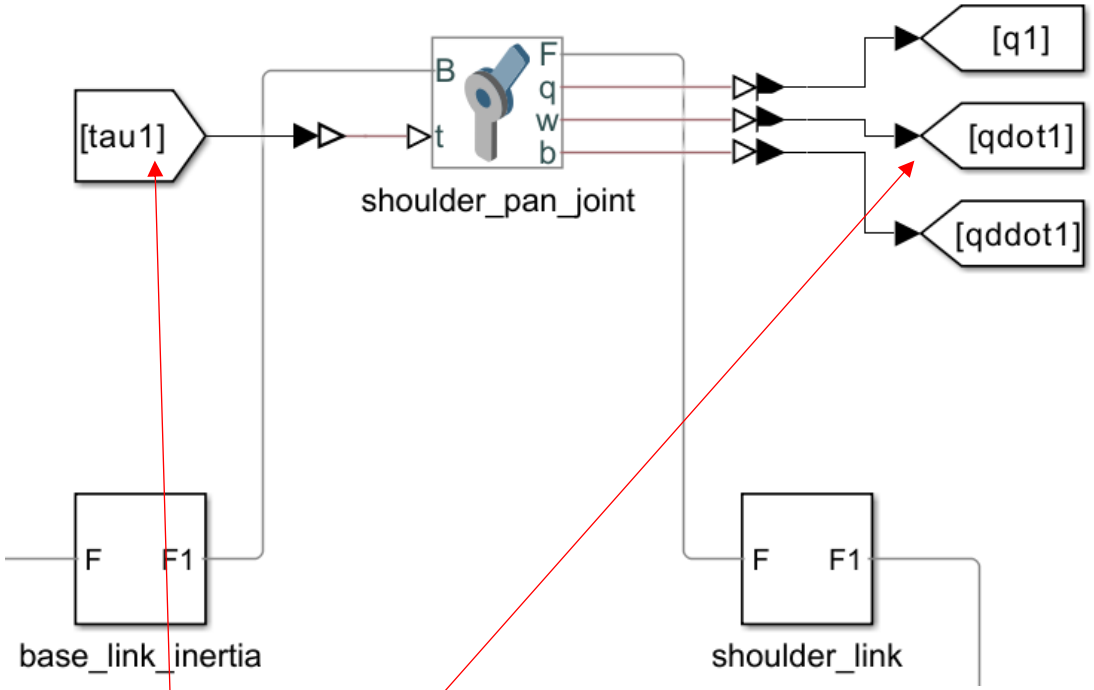


'UR10_control.slx'

UNIVERSITÀ DEGLI STUDI DI GENOVA

Using commercial robots

Once you get the multibody chain(Sim/Control .slx file), **configure inputs and outputs.**



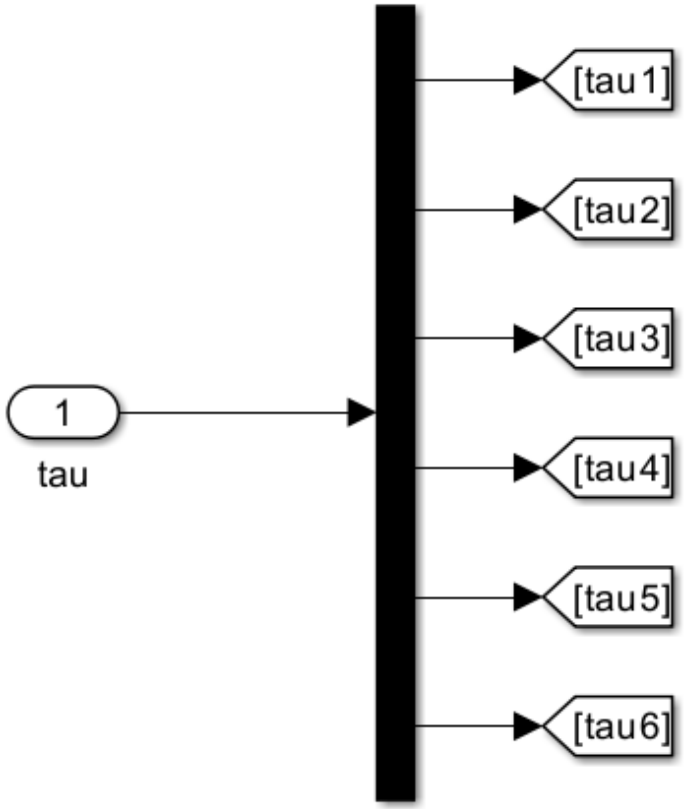
Use **'From'** and **'Goto'** Simulink blocks to move signals around a complex model, to avoid mess with too many cables and **keep your model clean**

Configure each joint to take external input torque and to measure its position, velocity and acceleration

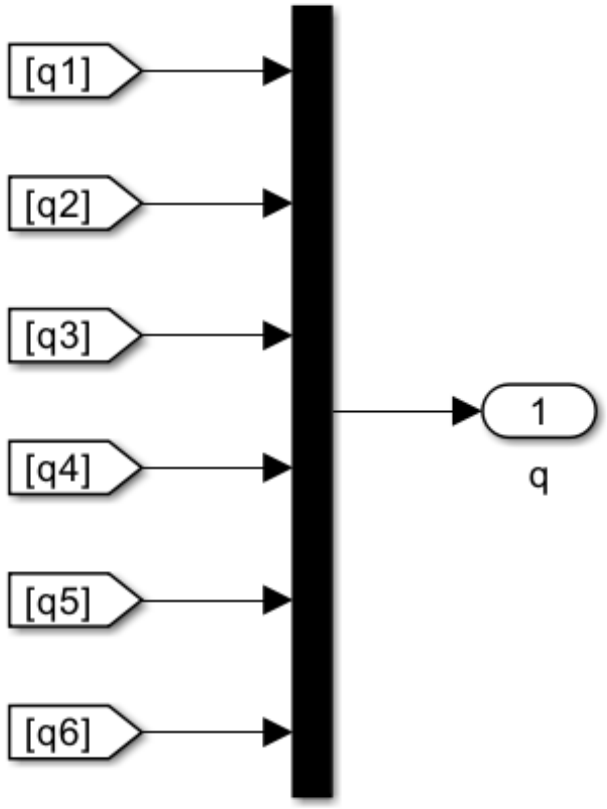
Actuation	
Torque	Provided by Input
Motion	Automatically Computed
Sensing	
☑ Position	
☑ Velocity	
☑ Acceleration	

Using commercial robots

Stack signals and **make the subsystem**



Use **Mux and Demux blocks** to **unpack or stack all the joint variables into vectors**



Robot System Input

Robot System Output

The PD Control for Regulation

Goal: **Reach a constant target joint configuration** (**tasks** like **lifting a load**, **holding a camera**, **holding** the **grasp on an object**)

Proportional Gain

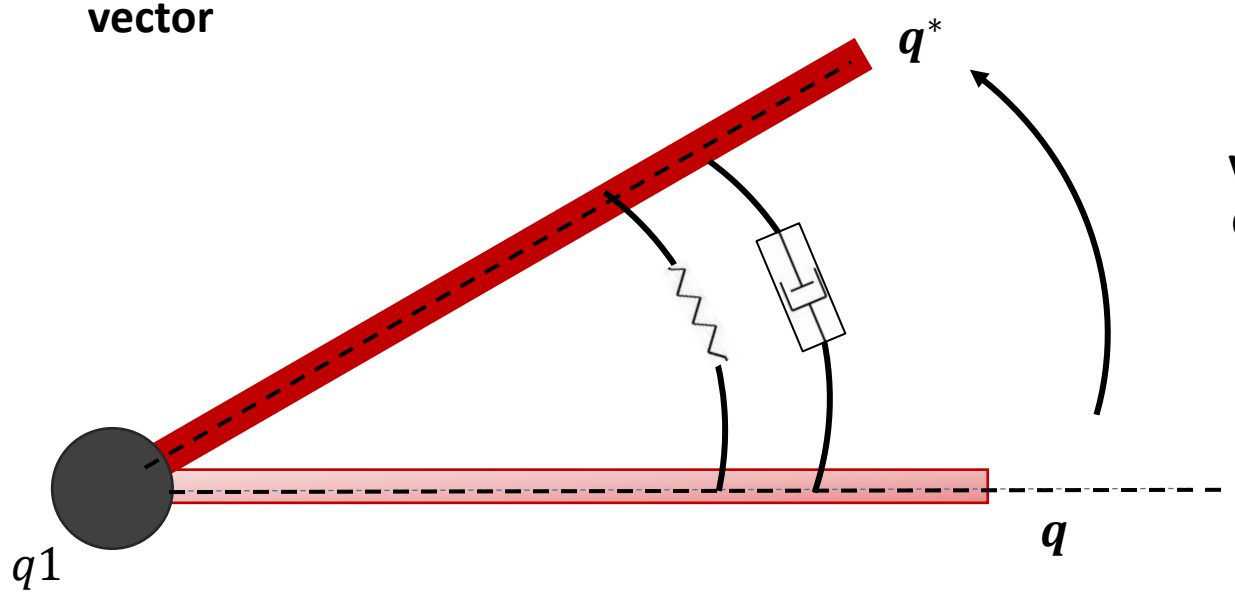
Derivative Gain

$$\tau = K_P(q^* - q) - K_D\dot{q}$$

Measured Joint Velocities

Control torque vector

Joint Position Error



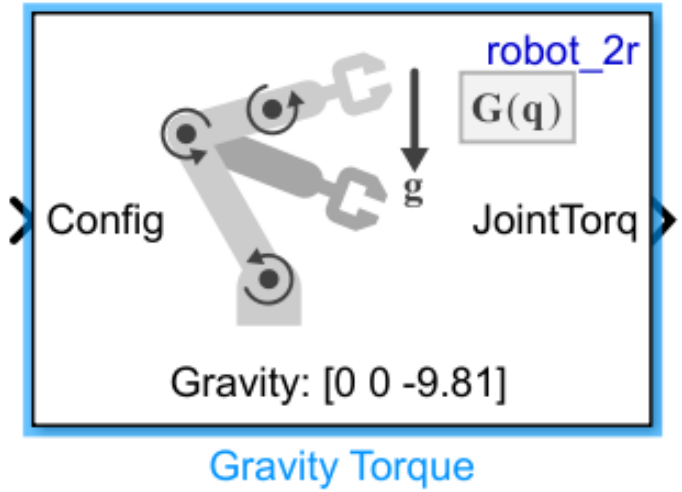
This control law can be **visualized** as a **virtual spring-damper system** which **drives the joint** towards its **target** while **damping oscillations**

PD Control + Gravity Compensation

Regulation control law with a **feedforward torque** to **cancel out gravity** effects. It can be obtained with algorithms developed in previous exercise sessions.

$$\tau = K_P(q^* - q) - K_D\dot{q} + \tau_g$$

For **complex structures** you can also use the Toolbox block '**Gravity Torque**'



Knowledge on the **dynamic model** is **not required**, **each joint** is **approximated as a linear decoupled system**

This controller exploits a **linear approximation of the robot**, **nonlinearities** and **couplings** which arise in the **full dynamic model** are **not considered.**

The Computed Torque Control

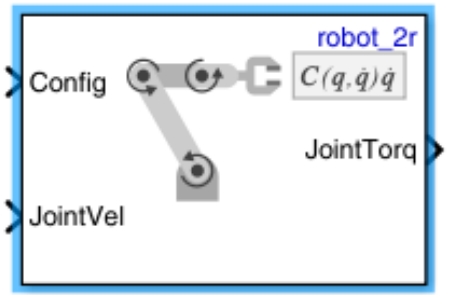
Goal: Follow (track) a desired joint trajectory $q_d(t)$ over time.

Tracking a **dynamic trajectory** requires **knowledge of couplings and nonlinearities given by** the **motion** of the **multibody structure**

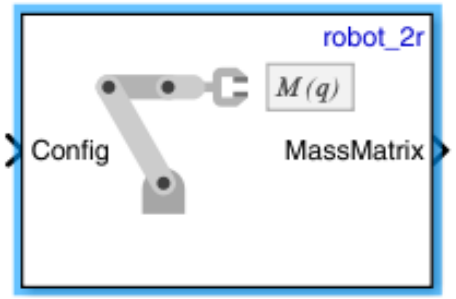
Given a **robot structure** and a **'rigidBodyTree'** entity, you can retrieve dynamic model terms with the following **blocks**

Dynamic Model

$$A(q)\ddot{q} + B(q,\dot{q})\dot{q} + C(q) = \tau$$



Velocity Product Torque



Joint Space Mass Matrix

Computed Torque Control Law

$$\tau_d = A(q)[\ddot{q}_d + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q})] + B(q,\dot{q})\dot{q} + C(q)$$

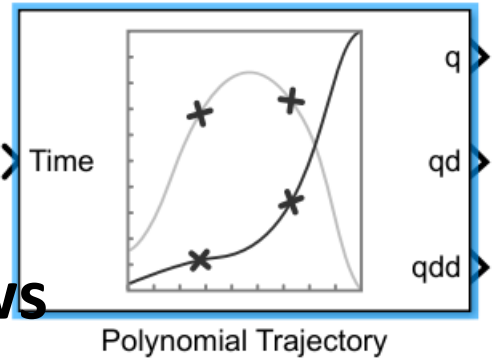
Mass Matrix — **Trajectory Tracking Controller** — **Velocity Product (Coriolis Torque)** — **Gravity Torque**

IMPORTANT: Be careful to the **notation mismatch.** In the course notation the Gravity term is called C, in the matlab notation that is the Coriolis Torque

UNIVERSITÀ DEGLI STUDI DI GENOVA

Exercises

Hint: To **simulate a twice-differentiable** joint **trajectory** use the polynomial trajectory block



- **Implement** the **PD + gravity and Computed Torque control laws** for the double pendulum and/or for the UR10 robot model

- **Evaluate** your implementation with **arbitrary joint reference positions (PD) and trajectories (Computed Torque)**. Remember to **always check the control errors**!

- Consider the **double pendulum** model, **implement the Computed Torque controller without the Toolbox blocks** by using the **Recursive Newton Euler** algorithm that you have implemented in Laboratory session 3.